Site reliability engineering

Define reliability with SLOs, respond to incidents, run on-call, plan capacity, practise failure and manage change safely.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. SLIs, SLOs and error budgets (16 min)
2. Incident response (16 min)
3. Blameless postmortems and toil (15 min)
4. Reliability patterns (17 min)
5. Running a healthy on-call rotation (15 min)
6. Capacity planning and performance testing (15 min)
7. Chaos engineering, game days and disaster recovery (15 min)
8. Change management, release engineering and progressive delivery (15 min)
9. Production readiness reviews and launch checklists (13 min)
10. Incident communication and stakeholder management (12 min)
11. Dependencies and third-party risk (12 min)
12. Disaster recovery exercises (13 min)
13. Debugging production under pressure (15 min)
14. Senior skills: writing design docs and RFCs (12 min)
15. Senior skills: estimating and planning work (11 min)
16. Senior skills: running incident calls and post-mortem meetings (12 min)
17. Senior skills: mentoring and growing other engineers (11 min)
18. Senior skills: influencing without authority and managing stakeholders (11 min)
19. Senior skills: interviewing and hiring as the interviewer (11 min)
Cheat sheet

Lesson 1. SLIs, SLOs and error budgets

16 minutes. Define reliability in numbers users care about, set targets that balance speed and stability, and use error budgets to make release decisions.

"The site should be reliable" is not a target. Nobody can tell whether it was met, whether it is worth more investment, or when to stop shipping features and fix things. Site reliability engineering gives reliability a number, an agreed target, and a rule for what to do when the target is at risk.

What you will learn

- SLIs, SLOs and SLAs, and how they differ
- Choosing good indicators
- Calculating an error budget
- Using the budget to guide decisions

Three terms

Term	Meaning	Example
SLI (indicator)	A measurement of service behaviour	Share of requests that succeed and finish within 300 ms
SLO (objective)	An internal target for an SLI over a window	99.9% of requests are good over 30 days
SLA (agreement)	A contract with consequences, usually financial	Credits if monthly availability drops below 99.5%

The SLO is stricter than the SLA, so you notice trouble before you owe a customer anything.

Choosing SLIs

Measure what users experience, from as close to the user as practical (load balancer or client, not an internal health check). A good SLI is a **ratio of good events to valid events**:

```
SLI = good requests / valid requests
```

Service type	Useful SLIs
Request-driven API	Availability (non-5xx ratio), latency (share faster than a threshold)
Data pipeline	Freshness (data newer than N minutes), correctness, completeness
Storage	Durability, availability
Batch job	Completed on schedule

Pick a few per service. Three to five is plenty, and focus on the **critical user journeys**, such as sign in, search and checkout, rather than every endpoint.

Setting the target

Higher is not always better. Each extra nine costs far more and slows delivery. Users' own devices and networks fail too, so past a point they cannot tell the difference.

Availability target	Downtime allowed per 30 days
99%	about 7 hours 12 minutes
99.9%	about 43 minutes
99.95%	about 21 minutes
99.99%	about 4 minutes

MARGA by EknathaLabs marga.eknathalabs.com

Calculate it yourself: a 30-day window has 43,200 minutes, and 0.1% of that is 43.2 minutes. Start from what users actually need, look at what you currently achieve, and set a target you can meet with reasonable effort, then tighten it deliberately.

The error budget

The **error budget** is the amount of unreliability the SLO allows:

```
error budget = 1 - SLO
```

For a 99.9% SLO over 30 days, you may fail 0.1% of requests. If you serve 10 million requests, that is 10,000 failures. The budget turns reliability into something you can spend:

- **Budget remaining**: ship features, run experiments, make risky changes.
- **Budget low or exhausted**: slow down. Prioritize reliability work, freeze risky releases, and fix what is burning the budget.

This is the heart of the idea: product and engineering share one number, so the argument "features versus stability" becomes a decision rule agreed in advance. Reliability is a feature that competes for the same effort.

Measuring it

```
# Availability SLI over 30 days
sum(rate(http_requests_total{service="shop-api", status!~"5.."}[30d]))
  /
sum(rate(http_requests_total{service="shop-api"}[30d]))

# Fraction of the error budget remaining (99.9% SLO)
1 - (
  (1 - (sum(rate(http_requests_total{service="shop-api", status!~"5.."}[30d]))
        / sum(rate(http_requests_total{service="shop-api"}[30d]))))
  / 0.001
)
```

Long-range queries like these are costly, so teams precompute them with recording rules. Display the remaining budget on the service dashboard where everyone can see it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How SLIs, SLOs and error budgets fits together

Part	What it does	What goes wrong
Three terms	The SLO is stricter than the SLA, so you notice trouble before you owe a customer anything.	Setting SLOs at 100% or at whatever the service happens to do today.
Choosing SLIs	Measure what users experience, from as close to the user as practical (load balancer or client, not an internal health check).	Measuring server health instead of user experience.
Setting the target	Higher is not always better. Each extra nine costs far more and slows delivery.	SLOs with no policy for what happens when the budget is spent.
The error budget	The error budget is the amount of unreliability the SLO allows.	Too many SLOs, none of them used to make decisions.
Measuring it	Long-range queries like these are costly, so teams precompute them with recording rules.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Three terms to Choosing SLIs / Choosing SLIs to Setting the target / Setting the target to The error budget / The error budget to Measuring it

MARGA by EknathaLabs marga.eknathalabs.com

Incident: the budget is gone on day twelve

> **Incident:** *The checkout SLO is 99.9% over 30 days. On day 12 the dashboard shows the budget at -20%: more failures have occurred than the whole month allows.*

The numbers say what must happen next, and the policy should already have said so. Look at what consumed it:

```
Budget spent (12 days):
  Release 3.2.0 rollback (day 4):      55%
  Payment provider timeouts (day 8):   40%
  Everything else:                     25%
```

Two events made up 95% of the loss. The agreed response is to pause non-urgent feature releases for the service, spend the sprint on the causes (a safer rollout with automatic canary abort, and timeout and fallback behaviour for the payment provider), and review again when the budget recovers as the window moves on. Importantly, the conversation was about data and a policy everyone had signed up to, not about whose fault it was. If the budget is *never* used, that is also a signal: the target may be too strict, and you could be shipping faster.

Predict the budget

Common mistakes

- Setting SLOs at 100% or at whatever the service happens to do today
- Measuring server health instead of user experience
- Too many SLOs, none of them used to make decisions
- SLOs with no policy for what happens when the budget is spent

Check yourself

What is an error budget?

Answer: The amount of unreliability the SLO allows, equal to 1 minus the SLO

Approximately how much downtime does 99.9% allow in 30 days?

Answer: 43 minutes

What should happen when the error budget is exhausted?

Answer: Slow risky releases and prioritize reliability work, as agreed in the SLO policy

Which is the best form of SLI?

Answer: Good events divided by valid events as experienced by users

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Incident response

16 minutes. Run incidents with clear roles, severity levels and communication, mitigate first and diagnose second, and hand over cleanly.

At 03:12 an alert fires. The next hour decides how much a problem costs. Teams that handle incidents well are not smarter than others. They rehearsed a process so nobody has to invent one under stress.

What you will learn

- Incident roles and who does what
- Severity levels
- The sequence: detect, triage, mitigate, resolve
- Communication during an incident

Roles

On anything beyond a trivial incident, split the work so one person is not debugging, deciding and answering questions at once.

Role	Responsibility
Incident commander (IC)	Runs the response, makes decisions, assigns work. Does *not* debug
Operations lead	Investigates and applies changes to the systems
Communications lead	Posts updates to stakeholders and customers on a fixed rhythm
Scribe	Records a timeline of what was seen and done, with timestamps

For small incidents one person may hold several roles, but the IC role should still be explicit: "I am IC." If the IC begins digging into the problem, they should hand command to someone else first.

Severity

Define levels in advance so nobody argues during the incident. An example scheme:

Level	Impact	Response
SEV1	Major outage or data loss; most users affected	Page everyone needed, all hands, updates every 15-30 minutes
SEV2	Significant degradation; some users or a key feature affected	Page on-call, updates every 30-60 minutes
SEV3	Minor impact; workaround exists	Handle in working hours

When in doubt, declare the higher level. It is cheap to downgrade and expensive to escalate late.

The sequence

1. **Detect.** An alert, a customer report or a colleague. Acknowledge it so others know someone owns it.
2. **Triage.** How bad is it, who is affected, is it getting worse? Assign severity and open a dedicated channel.
3. **Mitigate. Stop the harm first.** Roll back, fail over, disable a feature flag, shift traffic, scale up. Do not wait for the root cause.
4. **Diagnose.** Once users are no longer hurt, find out why, working from evidence.
5. **Resolve.** Confirm the metrics recovered and stayed recovered.
6. **Follow up.** Schedule the postmortem and track the actions.

MARGA by EknathaLabs marga.eknathalabs.com

> **Tip:** *Ask early: "What changed?" Most incidents trace back to a recent deployment, configuration change, dependency, or traffic pattern. Checking the deployment log and change calendar first finds many causes in minutes.*

Here is what that first check looks like in practice on a Kubernetes service:

```
$ kubectl rollout history deployment/shop-api -n shop | tail -3
$ kubectl get events -n shop --sort-by=.lastTimestamp | tail -8
$ kubectl get deploy shop-api -n shop -o jsonpath='{.metadata.annotations.deployment\.kubernetes\. \
    io/revision}'
```

If the newest revision was rolled out minutes before the alert fired, that is your first suspect, and `kubectl rollout undo deployment/shop-api -n shop` is the mitigation.

Prefer mitigations that are **reversible, well understood and fast**. A rollback to a known good version beats a clever hot fix written at 3 a.m.

Communication

- Use **one channel** for the incident and keep debate out of the status updates.
- Post updates on a **schedule**, even when the update is "no change, still investigating, next update at 03:45".
- State **impact, status and next step**: facts, not guesses.
- Keep a running **timeline** with timestamps for the postmortem.

A good update:

```
03:30 UTC - SEV2 - Checkout errors
Impact: about 15% of checkout attempts fail with an error page.
Status: Cause identified as release 3.2.0. Rollback in progress.
Next update: 03:45 UTC or sooner if status changes.
```

Handover and fatigue

Long incidents need fresh people. Hand over explicitly with a summary: what is broken, what has been tried, current theory, who is doing what. Rotate the IC after a few hours, and make sure people eat and rest. Tired people make mistakes that turn a bad night into a worse one.

Incident: two people fix the same thing differently

> **Incident:** *Checkout is failing. One engineer rolls back the release. Another, unaware, scales the database up. A third restarts the API Pods. Errors continue, and nobody can tell which action helped or hurt.*

Uncoordinated changes destroy your ability to learn what worked and can make things worse. The first action of a good IC is to establish control:

```
IC: I'm taking command of this incident. Freeze all changes unless I approve them.
IC: Ops lead is Dana. Dana, you make changes to production. Everyone else reports findings to the  \
    scribe.
IC: Hypothesis: release 3.2.0. Dana, roll back now. We will watch error rate for 5 minutes.
```

One change at a time, announced before it is made, with a metric to watch afterwards. The rollback restores service within four minutes, and because only one change was made, the cause is clear. The other changes (the database scale-up) are queued for review rather than left in place without understanding.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: The shape of an incident

Part	What it does	What goes wrong
Detect	An alert or a customer report starts the incident.	A slow detection means longer impact.
Triage and declare	Set severity, name an incident lead and open a channel.	Without a lead everyone investigates and nobody decides.
Mitigate	Restore service first: rollback, failover or disable a feature.	Digging for root cause first prolongs the outage.
Communicate	Regular updates to stakeholders and a status page.	Silence creates extra interruptions.
Review	A blameless review with action items and owners.	Without follow-up the same incident returns.

Flow: Detect to Triage and declare / Triage and declare to Mitigate / Triage and declare to Communicate / Mitigate to Review / Communicate to Review

Common mistakes

- Debugging before mitigating
- Several people changing production at once
- Silence: no updates for stakeholders
- No clear incident commander
- Running a long incident without handover or rest

Check yourself

What should happen first in most incidents?

Answer: Mitigate the impact, for example by rolling back

What is the incident commander's job?

Answer: Coordinate the response, make decisions and assign work

Why define severity levels beforehand?

Answer: So response expectations are agreed and not argued during the incident

What is a useful first diagnostic question?

Answer: What changed recently?

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Blameless postmortems and toil

15 minutes. Learn from incidents without blame, write postmortems that produce real fixes, and identify and eliminate toil.

An incident that teaches nothing will repeat. The postmortem turns a bad night into lasting improvement, but only if people feel safe describing what actually happened. And between incidents, the daily drudgery called toil quietly consumes the time you would need to prevent the next one.

What you will learn

- What blameless means and why it matters
- The structure of a useful postmortem
- Writing action items that get done
- Defining and reducing toil

Blameless does not mean accountability-free

Complex systems fail because of many contributing factors, not a single careless person. If people are punished for mistakes, they hide them, and the organization loses the knowledge it needs. **Blameless** means we ask "how did our systems, processes and tooling make this mistake easy and its impact large?" rather than "who did it?"

> **Why it matters:** *If an engineer ran a command that took down production, the useful question is why a single command could do that without a confirmation, a dry run, a limited blast radius or a review. Fix the system and the next person is protected, whoever they are.*

Accountability remains. It shows up as owned, tracked action items, not as individual blame.

Structure of a postmortem

Section	Content
Summary	Two or three sentences: what broke, for how long, who was affected
Impact	Users, requests, revenue, SLO budget consumed
Timeline	Timestamped events: detection, escalations, actions, recovery
Root causes and contributing factors	The chain of conditions that allowed the incident
What went well / badly / where we got lucky	Honest reflection, including luck
Action items	Concrete, owned, dated changes
Lessons learned	What others can learn

Write it within a few days while memories are fresh. For SEV1 and SEV2 incidents, hold a short review meeting, and publish the document widely so other teams learn from it.

Finding causes without stopping early

Use a "five whys" style chain, but treat it as a way to explore branches, not to find one "root cause".

```
Why did checkout fail?            The inventory service returned errors.
Why did it return errors?         Its database connection pool was exhausted.
Why was the pool exhausted?       A new query held connections for 6 seconds.
Why did the query take 6 seconds? A missing index on a new column.
Why did it reach production?      Load tests used a small dataset; no query-plan check in CI.
Why was there no alert earlier?   No saturation alert on connection pools.
```

Each answer reveals a gap you can close: a missing index, a test that did not reflect production data, and missing monitoring. Stopping at "a developer forgot an index" would have fixed nothing systemic.

MARGA by EknathaLabs marga.eknathalabs.com

Action items that get done

Weak: "Improve monitoring." Strong:

Action	Type	Owner	Due
Add an alert for connection pool saturation above 80% for 5 minutes	Detect	Dana	2026-11-07
Add a CI step that fails on queries with full table scans in test	Prevent	Arun	2026-11-21
Document the rollback steps in the runbook and rehearse once	Mitigate	Mei	2026-11-14

Prefer actions that prevent, detect or reduce impact over "be more careful", which is not a control. Track them in the same system as your regular work, and review completion. A postmortem whose actions are never done is theatre.

Toil

Toil is work that is manual, repetitive, automatable, tactical (reactive), without lasting value, and that grows as the service grows. Restarting a stuck job by hand every week, manually creating accounts, copying certificates: all toil.

Not toil: engineering projects, design, and the overhead of meetings and planning. Toil is specifically the repetitive operational work that a machine could do.

Reducing toil	Example
Automate the task	A script or controller replaces the manual procedure
Remove the cause	Fix the bug that makes the job stick, rather than restarting it
Self-service	Give developers a tool to do the task without a ticket
Stop doing it	Some tasks provide no value

A common guideline is to keep toil **below about half** of an operations engineer's time, and to put the rest into engineering that reduces future toil. Measure it: ask the team to tag time spent, and rank sources of toil by hours per month. Automate the biggest first, and where automation costs more than the toil will ever, do not.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Blameless postmortems and toil fits together

Part	What it does	What goes wrong
Blameless does not mean	Complex systems fail because of many contributing factors, not a single careless person.	Naming a person as the cause.
Structure of a postmortem	Write it within a few days while memories are fresh.	Stopping at one "root cause".
Finding causes without stopping	Use a "five whys" style chain, but treat it as a way to explore branches, not to find one "root cause".	Finding causes without stopping early Use a "five whys" style chain, but treat it as a way to explore branches, not to find one "root...
Action items that get done	Weak: "Improve monitoring." Strong.	Vague actions with no owner or date.
Toil	Toil is work that is manual, repetitive, automatable, tactical (reactive), without lasting value, and that grows as the service grows.	Letting toil crowd out all engineering time.

Flow: Blameless does not mean to Structure of a postmortem / Structure of a postmortem to Finding causes without stopping / Finding causes without stopping to Action items that get done / Action items that get done to Toil

Incident: the same outage three times

> **Incident:** *An expired TLS certificate takes down the customer portal. It is the third certificate expiry in 18 months. The previous two postmortems each recorded the action "be more careful with renewals".*

The action items never changed the system, so the system produced the same result. Review the earlier documents and look for what a real fix would have been:

- **Detect**: alert when any certificate expires in under 21 days, from an external check of the live endpoints
- **Prevent**: automate issuance and renewal so no human is in the loop
- **Reduce impact**: inventory every certificate and its owner

```
$ echo | openssl s_client -connect portal.example.com:443 -servername portal.example.com 2>/dev/nu \
    ll | openssl x509 -noout -enddate
notAfter=Nov 11 09:00:00 2026 GMT
```

A scheduled check like this one, run against every public hostname, gives early warning. Replace the vague action with the three concrete ones, give them owners and dates, and add the certificate renewal to the toil list with a target of zero manual steps. The fourth occurrence never happens.

Common mistakes

- Naming a person as the cause
- Stopping at one "root cause"
- Vague actions with no owner or date
- Writing the postmortem and never reading it again
- Letting toil crowd out all engineering time

Check yourself

What does blameless mean in a postmortem?

Answer: The focus is on how systems and processes allowed the failure, not on punishing individuals

Which action item is best?

Answer: Add an alert for pool saturation above 80%, owner Dana, due in two weeks

Which of these is toil?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Manually restarting a stuck job every week

Why is it a problem to stop at one root cause?

Answer: Incidents usually have several contributing factors, and fixing only one leaves gaps

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Reliability patterns

17 minutes. Apply timeouts, retries with backoff, circuit breakers, load shedding and capacity planning so one failing part does not take down the whole system.

Large systems are always partly broken. A disk fails somewhere, a dependency slows down, a deploy goes wrong. Reliable systems are not those where nothing fails, but those designed so that failures stay small. A handful of patterns do most of the work.

What you will learn

- Timeouts, retries and the retry storm
- Circuit breakers and bulkheads
- Load shedding and graceful degradation
- Capacity planning and testing failure

Timeouts

Every call over a network needs a **timeout**. Without one, a slow dependency holds your threads or connections indefinitely, and your service becomes slow, then dead, because of someone else's problem.

- Set timeouts shorter than the caller's own deadline. If the user's request has 2 seconds, do not make a 10-second call inside it.
- Propagate a **deadline** down the call chain, so downstream work is abandoned when the caller has already given up.

Retries, with care

Retrying helps with brief, transient failures. Done badly, it multiplies load on a struggling service and turns a hiccup into an outage.

Rules for safe retries:

Rule	Reason
Retry only **idempotent** operations (or use idempotency keys)	A retried payment must not charge twice
Use **exponential backoff**	Waiting 1s, 2s, 4s gives the system room to recover
Add **jitter** (randomness)	Prevents all clients retrying at the same instant
Cap attempts (two or three) and total time	Avoids runaway work
Use a **retry budget**	Allow retries as at most about 10% of traffic, so they cannot swamp the service

```
import random, time

def call_with_retries(fn, attempts=3, base=0.2, cap=2.0):
    for attempt in range(attempts):
        try:
            return fn()
        except TransientError:
            if attempt == attempts - 1:
                raise
            sleep = min(cap, base * (2 ** attempt))
            time.sleep(random.uniform(0, sleep))
```

Warning: *If every layer of a five-layer system retries three times, one failing call at the bottom becomes 3 to the power 5, or 243 calls. Retry at one layer, usually the one closest to the user, and not at every layer.*

MARGA by EknathaLabs
marga.eknathalabs.com

Circuit breakers

A **circuit breaker** watches calls to a dependency. When failures pass a threshold, it **opens** and fails fast instead of calling, giving the dependency time to recover and saving your resources. After a pause it lets a few trial requests through (**half-open**), and closes again if they succeed.

```
closed (normal) --failures exceed threshold--> open (fail fast)
open --after cool-down--> half-open (trial requests)
half-open --success--> closed      half-open --failure--> open
```

Pair a breaker with a **fallback**: a cached value, a default, or a reduced feature.

Bulkheads and graceful degradation

A **bulkhead** isolates resources, as a ship's compartments keep one leak from sinking it. Give each dependency its own connection pool or thread pool, so a slow recommendations service cannot use all your capacity and starve checkout.

Graceful degradation means a failure in a non-critical feature should not break the critical path. If recommendations fail, show the product without them. Decide in advance which features are essential and which are optional.

Load shedding and limits

When demand exceeds capacity, doing nothing makes every request slow and then fail. It is better to **shed load**: reject some requests quickly (HTTP 429 or 503 with a `Retry-After` header) so the rest succeed. Prioritize: keep checkout, shed bulk exports. Add **rate limits** per client so one noisy caller cannot harm others, and **queues with bounded length** so overload becomes visible rather than hidden latency.

Capacity and testing failure

- Know your limits with **load tests** that match production traffic shapes.
- Keep headroom: plan for peak plus the loss of one zone or instance group.
- Watch saturation trends, and add capacity before the limit, not at it.
- **Practise failure.** Game days and chaos experiments (kill an instance, add latency to a dependency, fail over a database) reveal what breaks while you are watching and ready, not at 3 a.m.

Start small: pick a hypothesis ("if the cache is unavailable, checkout still works, only slower"), limit the blast radius, run it in a quiet period, and stop immediately if customer impact appears.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Reliability patterns fits together

Part	What it does	What goes wrong
Timeouts	Every call over a network needs a timeout.	Retries at every layer, without backoff or jitter.
Retries, with care	Retrying helps with brief, transient failures.	Retrying non-idempotent operations.
Circuit breakers	A circuit breaker watches calls to a dependency.	No timeouts on outbound calls.
Bulkheads and graceful degradation	A bulkhead isolates resources, as a ship's compartments keep one leak from sinking it.	One shared pool for all dependencies.
Load shedding and limits	When demand exceeds capacity, doing nothing makes every request slow and then fail.	Load shedding and limits When demand exceeds capacity, doing nothing makes every request slow and then fail.
Capacity and testing failure	Know your limits with load tests that match production traffic shapes.	Never testing failure on purpose.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Timeouts to Retries, with care / Retries, with care to Circuit breakers / Circuit breakers to Bulkheads and graceful degradation / Bulkheads and graceful degradation to Load shedding and limits / Load shedding and limits to Capacity and testing failure

Incident: a retry storm after a short blip

> **Incident:** *The inventory database has a 20-second hiccup. Inventory recovers, but the system stays down for 40 minutes with the API at 100% CPU.*

```
$ kubectl top pod -n shop
NAME                   CPU(cores)   MEMORY(bytes)
shop-api-6d8f7c9b5-a   1990m        480Mi
```

Charting the request rate that reached inventory shows what happened:

```
sum(rate(http_requests_total{service="inventory"}[1m]))
```

The rate was about 900 per second before the blip and 11,000 per second during it. During the blip, every client retried immediately and without limit. Inventory's request rate rose twelvefold, so when the database came back it was instantly overwhelmed again, and again. The retries *were* the outage. Break the loop first: shed load at the edge or temporarily reduce traffic so inventory can recover (scaling the API further would only add more retrying clients). Then fix the design: exponential backoff with jitter, a cap of two attempts, a retry budget, a circuit breaker on the inventory client, and a fallback that serves cached stock levels. Rehearse the scenario in a game day by injecting latency into inventory and confirming the system stays stable.

Common mistakes

- No timeouts on outbound calls
- Retries at every layer, without backoff or jitter
- Retrying non-idempotent operations
- One shared pool for all dependencies
- Never testing failure on purpose

Check yourself

Why add jitter to retries?

Answer: It stops all clients from retrying at the same moment

What does an open circuit breaker do?

Answer: Fails calls immediately without contacting the failing dependency

What is a bulkhead?

Answer: Isolating resources per dependency so one failure cannot consume everything

What is the best response when demand far exceeds capacity?

Answer: Shed some load quickly so the remaining requests succeed

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Running a healthy on-call rotation

15 minutes. Design on-call schedules, escalation and handoffs, write alerts and runbooks people can act on, measure pager load and keep the rotation sustainable.

On-call is where reliability meets people. A rotation that wakes engineers every night burns them out, drives them to leave and ends up making the service less reliable. A good one pages rarely, gives the person on duty what they need, and treats every page as a defect to remove.

What you will learn

- Rotation design: size, handoff, escalation and compensation
- What deserves a page and what does not
- Runbooks and the first fifteen minutes of a page
- Measuring load and reducing it

Designing the rotation

- **Size**: with fewer than five or six people, shifts come around too often. Aim for one week on duty in six or more, with a secondary for backup
- **Follow the sun** where teams are spread across time zones, so nobody works nights routinely
- **Primary and secondary**: if the primary does not acknowledge in five minutes, the page goes to the secondary, then to a manager
- **Handoff**: a short written summary at each change of shift, covering open incidents, flaky alerts and planned risky changes
- **Fairness**: pay or time off for on-call work, and no heroics expected after a hard night

```
Handoff, week of 2 June
Open: checkout latency ticket OPS-412, mitigated, fix due Wednesday
Noisy: disk-usage alert on batch-3 fires nightly, ignore until OPS-418 lands
Risky: database failover test scheduled Thursday 10:00 UTC
Runbooks changed: payment-gateway-timeouts updated after last incident
```

What deserves a page

A page interrupts a human, possibly at night. It should meet all of these tests:

1. **Urgent**: waiting until morning makes it worse
2. **Actionable**: a person can do something now
3. **Symptom-based**: users are, or soon will be, affected
4. **Real**: a false alarm rate low enough that people trust it

Everything else is a ticket or a dashboard. Page on symptoms such as error rate, latency and burn rate against your SLO, not on causes such as CPU at 85 percent. Alerts for causes belong in lower-severity channels, used to diagnose rather than wake anyone.

```
# page: the error budget is being consumed 14x too fast, over two windows
(
  sum(rate(http_requests_total{job="shop-api",code=~"5.."}[5m]))
  / sum(rate(http_requests_total{job="shop-api"}[5m]))
) > (14.4 * 0.001)
and
(
  sum(rate(http_requests_total{job="shop-api",code=~"5.."}[1h]))
  / sum(rate(http_requests_total{job="shop-api"}[1h]))
) > (14.4 * 0.001)
```

MARGA by EknathaLabs
marga.eknathalabs.com

Runbooks

Each paging alert links to a runbook that works for a tired person who did not write the service. A good runbook has:

- What the alert means and what users experience
- The first three things to check, as copy-paste commands
- Mitigations ranked by safety, such as rollback, failover and scale-up
- Who to escalate to and when
- Links to dashboards and the previous incidents

```
$ kubectl get pods -n shop -l app=shop-api
$ kubectl rollout history deployment/shop-api -n shop | tail -3
$ kubectl rollout undo deployment/shop-api -n shop
$ curl -fsS https://status.example.com/api/v2/status.json | jq -r '.status.description'
```

Treat runbooks as code: keep them in version control, review them, and update them in the same change as the system they describe. Even better, automate the safe steps so the runbook says "run this command" or the system does it unattended.

The first fifteen minutes

1. **Acknowledge** so others know someone owns it
2. **Assess**: how bad, who is affected, getting worse?
3. **Escalate early** if you are unsure: declare an incident and call the secondary. Asking for help in the first five minutes is not weakness
4. **Mitigate** before diagnosing, using the runbook
5. **Communicate** on a fixed rhythm

Measuring and reducing load

Track per rotation: pages per shift, pages outside working hours, time to acknowledge, and the share of pages that needed no action. A common guideline from SRE practice is no more than two incidents per shift, since each needs time to handle and to follow up.

```
$ curl -s -H "Authorization: Token token=$PD_TOKEN" "https://api.pagerduty.com/analytics/raw/incid \
    ents" | jq '.data | length'
```

Hold a weekly review of every page: Was it actionable? Could automation have fixed it? Should the alert change? Delete or downgrade alerts that nobody acted on. Pages that repeat belong in the toil backlog with an owner.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Running a healthy on-call rotation fits together

Part	What it does	What goes wrong
Designing the rotation	Size: with fewer than five or six people, shifts come around too often.	Alerts with no runbook or no clear owner.
What deserves a page	A page interrupts a human, possibly at night.	Paging on causes such as CPU and memory instead of symptoms.
Runbooks	Each paging alert links to a runbook that works for a tired person who did not write the service.	Blaming the individual for missing a page in a noisy system.
The first fifteen minutes	Acknowledge so others know someone owns it.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Measuring and reducing load	Track per rotation: pages per shift, pages outside working hours, time to acknowledge, and the share of pages that needed no action.	Nobody reviewing pages, so noise only grows.

Flow: Designing the rotation to What deserves a page / What deserves a page to Runbooks / Runbooks to The first fifteen minutes / The first fifteen minutes to Measuring and reducing load

Incident: the alert that cried wolf

> **Incident:** *Over three months the team's pager has grown to 40 pages a week, mostly disk alerts at 80 percent that clear themselves. A real database failover alert fires at 02:10 and the on-call engineer, having been woken four times already that week by false alarms, silences it and goes back to sleep. Orders fail until 06:00.*

Alert fatigue made the real signal look like the usual noise. Fix the system rather than the person:

```
$ kubectl get prometheusrules -A -o json | jq -r '.items[].spec.groups[].rules[] | select(.alert)  \
    | .alert' | sort | uniq -c | sort -rn | head
```

List every alert, review which ones led to action, and delete or downgrade the rest. Replace cause-based alerts with SLO burn-rate alerts. Add automatic disk clean-up for the repeating one, set quiet hours only for non-urgent notifications, and make a rule that any alert ignored three times must be fixed or removed that week.

Common mistakes

- Paging on causes such as CPU and memory instead of symptoms
- Alerts with no runbook or no clear owner
- A rotation too small, so people are always on
- Nobody reviewing pages, so noise only grows
- Blaming the individual for missing a page in a noisy system

Check yourself

What makes an alert worthy of a page?

Answer: It is urgent, actionable, symptom-based and trustworthy

Why escalate early when unsure?

Answer: More help in the first minutes reduces time to mitigation

What should a runbook contain?

Answer: What the alert means, first checks, mitigations, escalation and links

What is the right response to alert fatigue?

Answer: Review pages weekly and remove or fix noisy alerts

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 6. Capacity planning and performance testing

15 minutes. Forecast demand, find a service's limits with load tests, plan headroom and N+1 redundancy, and connect capacity to cost and to autoscaling.

Running out of capacity is an avoidable outage. Having too much wastes money. Capacity planning is the discipline of knowing how much load your system can carry, how fast demand is growing and how long it takes to add more.

What you will learn

- Demand forecasting from real traffic data
- Load, stress and soak testing
- Headroom, N+1 and failure-domain planning
- Lead times, autoscaling and cost

Know your demand

Start with measurements: requests per second (peak, not average), growth over months, and seasonal events such as sales or month-end processing. Use Prometheus to see the trend:

```
max_over_time(sum(rate(http_requests_total{job="shop-api"}[5m]))[7d:5m])
```

```
predict_linear(node_filesystem_avail_bytes{mountpoint="/data"}[7d], 14 * 24 * 3600) < 0
```

The second query forecasts when a disk will fill, based on the last week of growth. Combine technical trends with business input: product launches, marketing campaigns and customer onboarding all change demand in ways history cannot show.

Find the limits

You cannot plan capacity without knowing what one unit can handle. Test it.

Test	Question it answers
Load	Does the system meet its SLO at the expected peak?
Stress	At what load does it break, and how does it fail?
Soak	Does it stay healthy for hours? (finds leaks)
Spike	Can it handle a sudden jump?

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 },
    { duration: '5m', target: 100 },
    { duration: '2m', target: 300 },
    { duration: '5m', target: 300 },
    { duration: '2m', target: 0 },
  ],
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<400'],
  },
};

export default function () {
  const r = http.get('https://staging.example.com/api/products');
  check(r, { ok: (res) => res.status === 200 });
}
```

MARGA by EknathaLabs marga.eknathalabs.com

Run load tests against an environment that resembles production, never against production data without protection. Watch the saturation signals during the test: CPU, memory, connection pools, queue depth and database load. The first resource to saturate is the bottleneck, and the number to remember is the throughput at which latency starts to exceed the SLO, not the point of total failure.

```
$ k6 run loadtest.js
$ kubectl top pods -n shop --sort-by=cpu | head -5
$ kubectl get hpa -n shop -w
```

Headroom and redundancy

If one instance handles 200 requests per second at the SLO, and peak demand is 900, you need five instances for the load. Add:

- **Headroom** for growth and bursts, commonly 20 to 30 percent
- **N+1 or N+2** so that losing one node or one zone does not take you over capacity
- **Failure-domain thinking**: with three zones, losing one leaves 67 percent of capacity, so each zone must be sized to carry half the peak alone

```
peak load            900 rps
per-instance limit   200 rps at SLO
needed for load      5 instances
with 25% headroom    6 instances
survive loss of 1 of 3 zones (2 zones carry all): 3 per zone = 9 instances
```

The last line shows why "survive a zone failure" is the most expensive requirement and deserves an explicit decision rather than an accident.

Lead time and autoscaling

Capacity must arrive before demand. Ask how long it takes to add some: autoscaling takes minutes, ordering hardware or a large cloud commitment takes weeks, and database sharding takes months. Plan based on the **longest lead time** among the resources a service depends on.

Autoscaling covers short-term variation, but it does not remove the need for planning: quotas, subnet addresses, database connections and downstream services all have limits that autoscaling will hit.

```
$ aws service-quotas get-service-quota --service-code ec2 --quota-code L-1216C47A --query 'Quota.V \
    alue'
$ kubectl describe quota -n shop
$ aws ec2 describe-subnets --subnet-ids subnet-0abc --query 'Subnets[0].AvailableIpAddressCount'
```

Capacity and cost

Review utilisation regularly. Persistently low usage means over-provisioning, and persistently high usage means risk. Use commitments such as reserved capacity or savings plans for the steady base, on-demand or spot for the variable part, and tag resources so cost is attributed to the teams who control it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Capacity planning and performance testing fits together

Part	What it does	What goes wrong
Know your demand	Start with measurements: requests per second (peak, not average), growth over months, and seasonal events such as sales or month-end processing.	Forecasting only from history, with no input from the business.
Find the limits	You cannot plan capacity without knowing what one unit can handle.	Planning for average load instead of peak.
Headroom and redundancy	If one instance handles 200 requests per second at the SLO, and peak demand is 900, you need five instances for the load.	No headroom or N+1, so one failure causes an overload.
Lead time and autoscaling	Capacity must arrive before demand. Ask how long it takes to add some: autoscaling takes minutes, ordering hardware or a large cloud commitment takes weeks, and database...	Ignoring quotas and address space limits.
Capacity and cost	Review utilisation regularly. Persistently low usage means over-provisioning, and persistently high usage means risk.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Know your demand to Find the limits / Find the limits to Headroom and redundancy / Headroom and redundancy to Lead time and autoscaling / Lead time and autoscaling to Capacity and cost

Incident: the launch day that overwhelmed the database

> **Incident:** *Marketing launches a campaign without telling engineering. Web servers autoscale from 10 to 60 instances within minutes, but the database hits its connection limit of 500, queries queue up, and every instance starts failing health checks. The site is down for the launch.*

Autoscaling the stateless tier moved the bottleneck to the shared database.

```
$ psql -h db.internal -c "select count(*) from pg_stat_activity;"
 count
-------
   498
$ psql -h db.internal -c "show max_connections;"
```

Immediate mitigation: cap the Deployment's maximum replicas, shed load with a queue or cached pages, and scale the database or add a connection pooler such as PgBouncer. Long term: load-test the whole path including dependencies, set autoscaling maximums from the weakest dependency, and add a launch checklist that gives engineering notice of expected traffic.

Common mistakes

- Planning for average load instead of peak
- Testing only the stateless tier
- No headroom or N+1, so one failure causes an overload
- Ignoring quotas and address space limits
- Forecasting only from history, with no input from the business

Check yourself

Why plan for peak instead of average load?

Answer: Failures happen when demand is highest

What does a soak test reveal?

Answer: Problems that appear over time, such as memory leaks

What is N+1 redundancy?

Answer: Enough capacity to keep serving after losing one unit

MARGA by EknathaLabs marga.eknathalabs.com

Why can autoscaling the web tier cause an outage?

Answer: It can overload a shared dependency, such as a database, that does not scale with it

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Chaos engineering, game days and disaster recovery

15 minutes. Practise failure on purpose with controlled experiments and game days, test backups and failover, and set recovery objectives you have actually proved.

Systems fail in ways nobody predicted, usually at the worst time. You can discover the weaknesses during a calm Tuesday afternoon with the whole team watching, or at 03:00 with customers watching. **Chaos engineering** and **game days** choose the first option.

What you will learn

- The chaos experiment loop and how to keep it safe
- Running a game day
- Tools for injecting failure in Kubernetes and cloud
- Disaster recovery: RTO, RPO and proving your backups

The experiment loop

Chaos engineering is not breaking things at random. It is the scientific method applied to production readiness.

1. **Define steady state**: measurable behaviour that means "healthy", such as the checkout success rate and p95 latency
2. **Form a hypothesis**: "If one API Pod is killed, checkout success stays above 99.9 percent"
3. **Limit the blast radius**: start small, in staging or on a slice of traffic, with a clear way to stop
4. **Inject the failure**
5. **Observe** against the steady state, and compare with the hypothesis
6. **Learn and fix**, then repeat with a larger failure

An experiment that confirms the hypothesis is a success: you have evidence of resilience. An experiment that fails is a bigger success, since you found the weakness cheaply.

Safety rules

- Have a **hypothesis, an owner and an abort condition** before starting (for example "stop if the error rate exceeds 2 percent")
- Announce the experiment, and avoid running during incidents, releases or peak traffic
- Make sure monitoring and the stop button work first
- Begin in non-production, then production with a small scope
- Never test what you could not recover from

Injecting failure

```
$ kubectl delete pod -l app=shop-api -n shop --field-selector=status.phase=Running --wait=false |  \
    head -1
$ kubectl get pods -n shop -w
$ kubectl drain worker-3 --ignore-daemonsets --delete-emptydir-data
$ kubectl uncordon worker-3
$ sudo tc qdisc add dev eth0 root netem delay 200ms loss 2%
$ sudo tc qdisc del dev eth0 root
$ stress-ng --cpu 4 --timeout 120s
```

Dedicated tools make experiments repeatable and safer. **Chaos Mesh** and **LitmusChaos** run experiments as Kubernetes resources, and cloud providers offer **AWS Fault Injection Service** and **Azure Chaos Studio**.

MARGA by EknathaLabs
marga.eknathalabs.com

```
apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: kill-one-api-pod
  namespace: chaos-testing
spec:
  action: pod-kill
  mode: one
  selector:
    namespaces: ["shop"]
    labelSelectors:
      app: shop-api
  duration: "60s"
```

Useful failures to rehearse: a Pod or node dying, a zone becoming unavailable, a dependency returning errors or becoming slow, DNS failing, a disk filling, a certificate expiring and an expired credential.

Game days

A **game day** is a scheduled exercise where the team responds to a simulated failure, practising the human side: detection, communication, escalation and runbooks.

1. Pick a scenario drawn from past incidents or from a risk review, for example "primary database fails over"
2. Assign roles: a facilitator who injects events, the responders, an observer taking notes
3. Run it: the facilitator reveals symptoms step by step, and responders use normal tools and runbooks
4. Debrief within a day: what worked, what was slow, what runbook was wrong?
5. Track fixes like any other work

The result is rarely a technical finding alone. You will learn that the runbook refers to a dashboard that no longer exists, or that only one person knows the failover procedure.

Disaster recovery

Two numbers define what you must achieve:

- **RTO** (recovery time objective): how long the service may be down
- **RPO** (recovery point objective): how much data you may lose, measured in time

A backup strategy is only as good as its **last successful restore test**.

Strategy	Typical RTO	Cost
Backup and restore	Hours	Low
Pilot light (core running, rest scaled down)	Tens of minutes	Medium
Warm standby	Minutes	Higher
Active-active multi-region	Near zero	Highest

```
$ aws rds describe-db-snapshots --db-instance-identifier shop-prod --query 'DBSnapshots[-1].[Snaps \
    hotCreateTime,Status]'
$ aws rds restore-db-instance-from-db-snapshot --db-instance-identifier shop-restore-test --db-sna \
    pshot-identifier shop-prod-2025-06-02
$ psql -h shop-restore-test.abc.eu-west-1.rds.amazonaws.com -U app -c "select count(*) from orders \
    ;"
$ aws rds delete-db-instance --db-instance-identifier shop-restore-test --skip-final-snapshot
```

Automate this restore test on a schedule and alert if it fails. Measure the actual time and compare it with the RTO.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Chaos engineering, game days and disaster recovery fits together

Part	What it does	What goes wrong
The experiment loop	Chaos engineering is not breaking things at random.	Game days that nobody follows up with fixes.
Safety rules	Have a hypothesis, an owner and an abort condition before starting (for example "stop if the error rate exceeds 2 percent").	Running experiments without a hypothesis or abort condition.
Injecting failure	Dedicated tools make experiments repeatable and safer.	Chaos testing before basic monitoring works.
Game days	A game day is a scheduled exercise where the team responds to a simulated failure, practising the human side: detection, communication, escalation and runbooks.	Stating RTO and RPO that have never been measured.
Disaster recovery	Two numbers define what you must achieve.	Treating a successful backup job as proof of restorability.

Flow: The experiment loop to Safety rules / Safety rules to Injecting failure / Injecting failure to Game days / Game days to Disaster recovery

Incident: the backup that did not restore

> **Incident:** *The primary database is corrupted by a faulty migration. The team starts the restore, and discovers the nightly backups have been empty files for five weeks because a credential expired and the job reported success anyway. The newest usable copy is from a month ago.*

The problem is untested backups and a job that ignored its own errors. Recovery uses whatever remains:

```
$ aws s3 ls s3://acme-backups/shop/ --human-readable | tail -5
$ aws rds describe-db-snapshots --snapshot-type automated --query 'DBSnapshots[].[DBSnapshotIdenti \
    fier,SnapshotCreateTime]' --output text | tail -3
```

Fall back to the cloud provider's automated snapshots and point-in-time recovery, accept the data loss that remains and tell stakeholders plainly. Then fix the process: backup jobs must fail loudly, a scheduled restore test must verify row counts, alerts must fire on backup size anomalies, and a quarterly game day must exercise the full restore.

Common mistakes

- Running experiments without a hypothesis or abort condition
- Chaos testing before basic monitoring works
- Treating a successful backup job as proof of restorability
- Game days that nobody follows up with fixes
- Stating RTO and RPO that have never been measured

Check yourself

What is the first step of a chaos experiment?

Answer: Define steady state and form a hypothesis

What is the purpose of a game day?

Answer: Practise detection, communication and recovery as a team under a simulated failure

What does RPO describe?

Answer: How much data loss is acceptable, measured as time

How do you know a backup works?

Answer: You regularly restore it and verify the data

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Change management, release engineering and progressive delivery

15 minutes. Make changes safe and boring with small batches, feature flags, canary analysis, automated rollback and error-budget-aware release policies.

Most incidents begin with a change: a deployment, a configuration edit, a new dependency version, a migration. The goal of release engineering is not to stop change, because change is how a business improves, but to make each change small, observable and reversible.

What you will learn

- Why small, frequent changes are safer than rare large ones
- Feature flags and dark launches
- Canary releases with automated analysis and rollback
- Change windows, freezes and tying releases to the error budget

Small batches

A release of one change has one suspect when something breaks. A release of two hundred changes has two hundred. Frequent small releases make failures easy to locate and easy to revert. This is why teams with strong delivery practices both deploy more often **and** have lower failure rates.

Track four delivery measures (the DORA metrics): **deployment frequency**, **lead time for changes**, **change failure rate** and **time to restore service**. Improving the first two should not worsen the last two, and if it does, the safety net needs work.

Feature flags

A **feature flag** separates deploying code from releasing a feature. Code ships turned off, then is enabled gradually for a team, a percentage of users or a region, and can be switched off instantly without a deployment.

```
def checkout(cart, user):
    if flags.enabled("new-payment-flow", user=user.id, default=False):
        return new_payment_flow(cart)
    return legacy_payment_flow(cart)
```

```
flags:
  new-payment-flow:
    enabled: true
    rollout:
      percentage: 5
      allow: ["internal-staff"]
    owner: payments-team
    expires: 2025-09-30
```

Flags have costs: each one doubles the code paths to test. Give every flag an owner and an expiry date, and delete flags promptly after full rollout. Make sure the **default** is the safe behaviour when the flag service is unreachable.

Canary and progressive delivery

A **canary** sends a small share of traffic to the new version and compares it with the stable version before continuing.

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: shop-api
  namespace: shop
spec:
  replicas: 10
  selector:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    matchLabels:
      app: shop-api
  strategy:
    canary:
      steps:
        - setWeight: 5
        - pause: { duration: 5m }
        - analysis:
            templates:
              - templateName: error-rate-and-latency
        - setWeight: 25
        - pause: { duration: 10m }
        - setWeight: 50
        - pause: { duration: 10m }
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      containers:
        - name: api
          image: registry.example.com/shop/api:1.8.3
```

An analysis step queries your metrics, for example "5xx rate of the canary is not more than 1.2 times the baseline and p95 latency is under 400 ms". If the check fails, the rollout **aborts and reverts automatically**. This depends entirely on trustworthy metrics, so build the SLO dashboards first.

```
$ kubectl argo rollouts get rollout shop-api -n shop --watch
$ kubectl argo rollouts abort shop-api -n shop
$ kubectl argo rollouts undo shop-api -n shop
```

Database and configuration changes

Schema changes are the hardest to roll back. Use **expand and contract**: add new columns or tables first (compatible with old code), deploy the code that uses them, migrate the data, and only later remove the old structure. Never ship a migration and the code requiring it as one irreversible step. Treat configuration as code: reviewed, versioned, validated, and rolled out through the same pipeline and canary process.

Change policy and the error budget

Tie release behaviour to your reliability data:

- While the SLO is met, ship freely
- When the error budget is nearly spent, slow down, reduce risk and prioritise reliability work
- When it is exhausted, freeze non-essential changes until it recovers

A freeze at peak business times (a sales event, month-end) is reasonable, but a freeze is a blunt tool. Prefer to make releases safer over making them rarer.

Peer review and risk

Review proportionate to risk. A copy change needs less ceremony than a database migration. Ask of every risky change: What is the blast radius? How will we know it is wrong? How do we undo it, and how long does that take? Record the answers in the change description.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Change management, release engineering and... fits together

Part	What it does	What goes wrong
Small batches	A release of one change has one suspect when something breaks.	Large, infrequent releases bundling many changes.
Feature flags	A feature flag separates deploying code from releasing a feature.	Feature flags with no owner or expiry.
Canary and progressive delivery	A canary sends a small share of traffic to the new version and compares it with the stable version before continuing.	Canaries judged by gut feeling instead of metrics.
Database and configuration changes	Schema changes are the hardest to roll back.	Configuration changes that bypass all release controls.
Change policy and the error budget	Tie release behaviour to your reliability data.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Peer review and risk	Review proportionate to risk. A copy change needs less ceremony than a database migration.	Irreversible database migrations coupled to code deploys.

Flow: Small batches to Feature flags / Feature flags to Canary and progressive delivery / Canary and progressive delivery to Database and configuration changes / Database and configuration changes to Change policy and the error budget / Change policy and the error budget to Peer review and risk

Incident: the friday afternoon config change

> **Incident:** *At 16:40 on Friday an engineer edits a timeout value in a shared configuration service for "a quick fix". It applies to every service at once. Within minutes, requests to the payment provider time out across the platform, and nobody can immediately say what changed.*

No change record, no gradual rollout, no quick way to see recent changes:

```
$ kubectl get events -A --sort-by=.lastTimestamp | tail -10
$ kubectl rollout history deployment/shop-api -n shop
$ curl -s https://config.example.com/api/audit?since=30m | jq -r '.[] | [.time,.user,.key,.old,.ne \
    w] | @tsv'
```

The audit trail reveals the edit. Revert the value, which restores service, and then add safeguards: configuration changes go through the same review, canary and rollback tooling as code; recent changes are shown on every dashboard as annotations; and risky changes avoid Friday afternoons unless they are fixing an incident.

Common mistakes

- Large, infrequent releases bundling many changes
- Feature flags with no owner or expiry
- Canaries judged by gut feeling instead of metrics
- Configuration changes that bypass all release controls
- Irreversible database migrations coupled to code deploys

Check yourself

Why are small frequent releases safer?

Answer: Each change has fewer suspects and is easier to revert

What does a feature flag separate?

Answer: Deploying code from releasing a feature to users

What should happen when canary analysis fails?

Answer: The rollout aborts and reverts automatically

What does the expand and contract pattern do?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: Makes schema changes compatible with old and new code so they can be rolled back

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Production readiness reviews and launch checklists

13 minutes. Check that a service is ready for real users before launch, covering ownership, SLOs, monitoring, capacity, failure modes, rollout, runbooks and security, with a lightweight review that teams value.

Many incidents are not surprises. They come from services that launched without alerts, without a rollback plan, without anyone who knew how to restart them, or without ever being load tested. A production readiness review (PRR) is a structured conversation that looks for those gaps before users find them. Done well, it helps teams; done badly, it becomes a hated form.

What you will learn

- What a PRR is and when to hold one
- A practical checklist by area
- How to run the review so it helps
- Launch criteria and follow-up
- Keeping a service ready after launch

When

Hold a review before a new service takes production traffic, before a big change in traffic or architecture, and when ownership moves between teams. Start early, since findings are cheaper to fix when the design is still flexible. Keep it proportional: a small internal tool needs a light check; a payment service needs a thorough one.

Checklist by area

Ownership and support

- A named owning team and an on-call rotation with an escalation path
- A service catalogue entry with contacts, repository, dashboards and runbooks
- Agreed support hours and severity definitions

Reliability targets

- SLIs and SLOs written down, with an error budget policy
- Known dependencies, and what happens when each one fails (timeouts, fallbacks, circuit breakers)

Monitoring and alerting

- Dashboards with traffic, errors, latency and saturation
- Alerts on symptoms users feel, each with a runbook link
- Logs and traces with request ids

Capacity and performance

- Load test results against expected peak plus headroom
- Autoscaling tested; limits and quotas checked
- Behaviour under overload: shed load or degrade, not collapse

Change and release

- Automated build and deploy, with canary or staged rollout
- Rollback tested and fast; database changes backwards compatible
- Feature flags for risky behaviour

MARGA by EknathaLabs
marga.eknathalabs.com

Failure and recovery

- Backups exist and a restore has been tried
- Single points of failure listed with decisions
- Behaviour on zone loss known; recovery steps documented

Security and data

- Least-privilege access, secrets in a manager, dependencies scanned
- Data classification, retention and privacy requirements met
- Audit logs for sensitive actions

Documentation

- Architecture diagram, runbooks for the top failure modes, and a launch plan with a rollback trigger

Running the review

Make it a conversation of an hour or two with the team and one or two experienced reviewers from outside, using the checklist as prompts. Ask "what happens when X fails?" rather than "do you have X?". Record findings with an owner and severity: must fix before launch, should fix within a month, or accepted risk (with the name of who accepted it). Share the checklist ahead of time so the team can prepare, and treat the review as help, not an audit.

After launch

Readiness decays. Re-review yearly, after major incidents, and when the service changes a lot. Use postmortem actions to update the checklist: every incident that a review could have found is a missing question.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Production readiness reviews and launch checklists fits together

Part	What it does	What goes wrong
When	Hold a review before a new service takes production traffic, before a big change in traffic or architecture, and when ownership moves between teams.	Treating the review as paperwork.
Checklist by area	Ownership and support.	Reviewing too late to change anything.
Running the review	Make it a conversation of an hour or two with the team and one or two experienced reviewers from outside, using the checklist as prompts.	No owner for follow-up items.
After launch	Readiness decays. Re-review yearly, after major incidents, and when the service changes a lot.	Never revisiting after launch.

Flow: When to Checklist by area / Checklist by area to Running the review / Running the review to After launch

Incident: the launch that nobody could roll back

> **Incident:** *A new recommendation service launches on a Friday and doubles page latency. The team cannot roll back because the database migration removed a column the old version needs.*

```
$ ./deploy rollback recs
error: migration 0042 is not reversible
```

No review had asked how to roll back. The team restored the column from a backup, which took five hours. Afterwards they added "rollback tested" and "migrations compatible" to the checklist, required a

launch plan with a rollback trigger, and stopped Friday launches without a reviewed plan.

Common mistakes

- Treating the review as paperwork
- Reviewing too late to change anything
- No owner for follow-up items
- Never revisiting after launch

Check yourself

When is the best time to start a production readiness review?

Answer: Early, while the design can still change

Which question is more useful in a review?

Answer: What happens when your main dependency becomes slow?

What should happen with a finding that the team decides not to fix?

Answer: It is recorded as an accepted risk with an owner's name

Why re-review a service after launch?

Answer: Systems and traffic change, and readiness decays

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Incident communication and stakeholder management

12 minutes. Communicate during outages with clear status updates, severity levels, an internal channel structure, customer messaging and a calm handover, so technical work is not slowed by confusion.

During an incident, the technical fix is only half of the job. Customers want to know whether they are affected, executives want to know the impact, other teams need to know whether to act, and responders need quiet. Poor communication prolongs outages, damages trust and burns out the people fixing the problem. Good communication is a skill you can practise like any other.

What you will learn

- Severity levels and what each triggers
- Separating the response channel from the updates channel
- The shape of a good status update
- Customer-facing messages and status pages
- Handovers, long incidents and the aftermath

Severity levels

Define levels by impact, not by cause, and say what each level triggers (who is paged, who must join, how often updates go out).

Level	Example impact	Response
Sev 1	Core service down for many users, data loss, security breach	Page everyone needed; commander assigned; updates every 30 minutes
Sev 2	Major feature degraded for some users	On-call plus owners; updates every hour
Sev 3	Minor issue with a workaround	Normal working hours; ticket

Anyone may raise severity; lowering it should be a decision of the incident commander. If unsure, start high and reduce.

Channels

Use a dedicated incident channel for responders, with a pinned summary: what is happening, who is commander, where the dashboard and the document are. Keep side conversations out. A separate updates channel (or a mailing list) carries the periodic summaries for everyone else, so executives and support do not interrupt responders to ask for news. The communications lead owns that channel and the status page, so the commander and the engineers can concentrate.

A good status update

Short, factual and on a schedule, even when there is nothing new ("no change, next update at 15:30"). Include:

1. **Impact:** who is affected and how, in plain words.
2. **Status:** what we know and what we are doing.
3. **Next update:** a time.

> *14:20 UTC: Some customers cannot complete payments (about 30 percent of attempts fail). We have identified a faulty change released at 13:50 and are rolling it back. Next update at 14:50 or sooner.*

Avoid guesses about the cause, internal jargon, blame, and promises about when it will be fixed unless you are sure. Do not paste raw logs.

MARGA by EknathaLabs marga.eknathalabs.com

Customers

Post on the status page early, within minutes of confirming impact. Say what customers can do (a workaround, retry later). When resolved, say so and what happens next. For serious incidents, follow with a plain account of what happened and what is being changed. If legal or contract rules apply, for example data breach notification, involve the right people early.

Long incidents and handover

After a few hours, people make mistakes. Rotate roles, hand over with a written summary (state, hypotheses tried, pending actions), and let people rest. The commander should step back from debugging, and a fresh commander can take over with a clean view.

Afterwards

Send a final summary, thank those involved, schedule the postmortem within a few days while memory is fresh, and track actions to completion. People who handled an incident well in difficult conditions deserve to hear it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Incident communication and stakeholder management fits together

Part	What it does	What goes wrong
Severity levels	Define levels by impact, not by cause, and say what each level triggers (who is paged, who must join, how often updates go out).	Everyone talking in the response channel.
Channels	Use a dedicated incident channel for responders, with a pinned summary: what is happening, who is commander, where the dashboard and the document are.	A separate updates channel (or a mailing list) carries the periodic summaries for everyone else, so executives and support do not interrupt...
A good status update	Short, factual and on a schedule, even when there is nothing new ("no change, next update at 15:30").	No update schedule.
Customers	Post on the status page early, within minutes of confirming impact.	No handover in long incidents.
Long incidents and handover	After a few hours, people make mistakes.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Afterwards	Send a final summary, thank those involved, schedule the postmortem within a few days while memory is fresh, and track actions to completion.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Severity levels to Channels / Channels to A good status update / A good status update to Customers / Customers to Long incidents and handover / Long incidents and handover to Afterwards

Incident: the silent outage

> **Incident:** *A checkout bug affects customers for 90 minutes. Engineers fix it quickly, but support learns about it from angry social media posts.*

```
14:05 detected   14:12 fix started   15:35 resolved
status page: first update at 15:50
```

The team had no communications role, the status page needed approval from someone who was asleep, and nobody told support. Afterwards they named a communications lead in the on-call rota, allowed the on-call engineer to post an initial status message, created templates for first updates, and added support to the updates channel.

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- No update schedule
- Speculating about the cause or blaming a person
- Everyone talking in the response channel
- No handover in long incidents

Check yourself

What three things should a status update include?

Answer: Impact, current status, and when the next update comes

Why have a communications lead during a serious incident?

Answer: Engineers can focus while stakeholders still get regular updates

When should the first customer-facing update go out?

Answer: As soon as impact is confirmed, within minutes

What helps in an incident longer than a few hours?

Answer: Rotating roles with a written handover

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Dependencies and third-party risk

12 minutes. Understand how your reliability depends on other services, set realistic targets, and plan for their failures.

Your service has a 99.95 percent target. It calls a payment provider with a 99.9 percent target, a search service at 99.9 percent and a messaging service at 99.5 percent. Your target is higher than some of your own dependencies. Reliability maths, and plain honesty about dependencies, change what promises you can make.

What you will learn

- How dependencies limit the availability you can offer
- Map hard and soft dependencies
- Mitigations: timeouts, fallbacks, caching, queues and redundancy
- Managing third-party providers: contracts, status, and exit plans

The arithmetic of dependencies

If a request needs services A and B, and both must work, the availability is the product: 99.9% x 99.9% = 99.8%. With five such dependencies at 99.9 percent, you cannot offer better than about 99.5 percent, however good your own code is. This is why every hard dependency must be visible in your target: either lower the promise, reduce the number of hard dependencies, or add a way to work when one fails.

Dependencies in the request path	Each at 99.9%	Combined
1	99.9%	99.9%
3	99.9%	99.7%
5	99.9%	99.5%

Draw the dependency map

List what a request needs: databases, caches, internal services, cloud services, DNS, identity providers, payment and email vendors. Mark each as **hard** (the request fails without it) or **soft** (the request still gives a useful answer). For each, note the owner, the target they publish, and how you would notice a failure. Include the hidden ones: DNS, certificates, time sync, the CI system you need to deploy a fix.

Make soft what you can

Turn hard dependencies into soft ones where the product allows it.

- **Timeouts and budgets**: a call must fail quickly so that it does not use up your own capacity.
- **Fallbacks**: show cached prices, or hide recommendations, when the recommender is down.
- **Queues**: accept an order now and send the confirmation email later.
- **Circuit breakers**: stop calling a failing dependency for a short time, then probe again.
- **Redundancy**: a second provider for critical functions such as payments or email, with a tested switch.

Each fallback must be tested, because an untested fallback usually fails during the first real outage.

Third-party providers

For external vendors, read the SLA: what is measured, what credit you get, and what is excluded. A credit is not compensation for your lost customers. Subscribe to the provider status page and alert on it, but also measure the dependency yourself, because status pages lag. Keep an exit plan for critical vendors: data export, a tested alternative, and a decision on how long an outage you would tolerate before switching. Review dependency health in your regular reliability review.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Dependencies and third-party risk fits together

Part	What it does	What goes wrong
The arithmetic of dependencies	If a request needs services A and B, and both must work, the availability is the product: 99.9% x 99.9% = 99.8%.	Promising more than the product of your hard dependencies.
Draw the dependency map	List what a request needs: databases, caches, internal services, cloud services, DNS, identity providers, payment and email vendors.	Forgetting hidden dependencies such as DNS and identity.
Make soft what you can	Turn hard dependencies into soft ones where the product allows it.	Fallbacks that were never tested.
Third-party providers	For external vendors, read the SLA: what is measured, what credit you get, and what is excluded.	Trusting a vendor status page alone.

Flow: The arithmetic of dependencies to Draw the dependency map / Draw the dependency map to Make soft what you can / Make soft what you can to Third-party providers

Incident: Our 99.95 depended on a 99.5

Incident: *A product promised customers 99.95 percent availability, then missed it every quarter.*

```
$ ./dependency-report.sh --quarter Q4
service        role   target   measured
payments-api   hard   99.9     99.82
search         soft   99.9     99.71
mail-gateway   hard   99.5     99.40
sms-vendor     hard   99.5     99.20
```

Three hard dependencies had lower targets than the promise. The review led to three changes: the email and SMS calls moved to a queue, so they became soft; search results fell back to a cached list; and the public target changed to 99.9 percent for the checkout path. The team measured each dependency from their own side, not only from provider status pages.

Common mistakes

- Promising more than the product of your hard dependencies
- Forgetting hidden dependencies such as DNS and identity
- Fallbacks that were never tested
- Trusting a vendor status page alone

Check yourself

What is the combined availability of two required services at 99.9 percent each?

Answer: About 99.8 percent

What is a soft dependency?

Answer: One where the request still gives a useful answer if it fails

What does a circuit breaker do?

Answer: Stops calling a failing dependency for a while, then probes again

Why measure a vendor yourself?

Answer: Status pages lag and may not show the failure your users see

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Disaster recovery exercises

13 minutes. Plan, run and learn from recovery drills: RTO and RPO, scenarios, safe execution and fixing what the drill reveals.

The recovery plan says "restore from backup in the secondary region". Nobody has done it, the credentials for the secondary account live on a laptop, and the backup tool changed its version last year. A plan that has not been exercised is a hope. Drills turn it into a capability and show the gaps while the stakes are low.

What you will learn

- Recovery objectives: RTO, RPO and what they cost
- Types of exercises: tabletop, partial, and full failover
- Run a drill safely, with roles and a stop rule
- Turn findings into fixes

Objectives first

RTO (recovery time objective) is how long you can be down. **RPO** (recovery point objective) is how much data you can lose, measured in time. A payment system may need an RTO of 15 minutes and an RPO of near zero, which needs replication and an automated failover. An internal report may accept an RTO of a day and an RPO of 24 hours, which a nightly backup can meet. Agree on objectives with the business per service, and write down what each one costs, because lower numbers mean more infrastructure and more practice.

Kinds of exercises

Start small and grow:

1. **Tabletop**: a group talks through a scenario ("the primary region is gone") and follows the plan on paper. It finds missing steps, contacts and decisions cheaply.
2. **Component restore**: restore a database from backup into a clean environment, and check the data and the time taken.
3. **Partial failover**: move one non-critical service to the secondary region during working hours.
4. **Full failover**: switch real production traffic. Do this only when earlier steps are boring.

Run them regularly (for example once or twice a year for the important systems), and after big changes.

Run it safely

Treat a drill like a change. Define the scenario and the success criteria (for example RTO 30 minutes, RPO 5 minutes), name a lead, a scribe and observers, and write down a **stop rule**: conditions under which you abort and go back. Announce the window to support and partners. During the drill, record timestamps for each step: detection, decision, start of recovery, first successful request, full service. Do not help the team with knowledge that a real responder would not have, or you will hide the gaps you came to find.

Learn and fix

The most valuable output is a list of surprises: an expired credential, a missing DNS record, a runbook step that needs a person who is on holiday, a restore that took four hours not one. Write them up blamelessly, give each one an owner and a date, and repeat the exercise to prove the fixes. Track measured recovery time and data loss against the objectives over time. If you cannot meet an objective, either invest to improve the system or change the objective openly.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Disaster recovery exercises fits together

Part	What it does	What goes wrong
Objectives first	RTO (recovery time objective) is how long you can be down.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Kinds of exercises	Start small and grow.	Never exercising the plan.
Run it safely	Treat a drill like a change. Define the scenario and the success criteria (for example RTO 30 minutes, RPO 5 minutes), name a lead, a scribe and observers, and write...	Helping the team during the drill so gaps stay hidden.
Learn and fix	The most valuable output is a list of surprises: an expired credential, a missing DNS record, a runbook step that needs a person who is on holiday, a restore that took...	Success criteria that are not measurable.

Flow: Objectives first to Kinds of exercises / Kinds of exercises to Run it safely / Run it safely to Learn and fix

Incident: The restore that took two days

Incident: *After a storage failure in a test environment, restoring the main database from backup took two days instead of the planned two hours.*

```
$ time pgbackrest --stanza=main restore
...
real    1612m41s
$ pgbackrest info | grep "database size"
database size: 4.1TB
```

The plan had been written when the database was 300 GB. It had grown to 4.1 TB, and the restore went through a single slow link. The team added regional restore targets with faster disks, switched to incremental backups with delta restore, and introduced a quarterly timed restore with an alert if it grows beyond 80 percent of the RTO. They also moved the credentials for the secondary account into the secret manager that works during a primary region outage.

Common mistakes

- Never exercising the plan
- Success criteria that are not measurable
- Helping the team during the drill so gaps stay hidden
- Not fixing the findings, or not repeating the test

Check yourself

What does RPO measure?

Answer: How much data you can afford to lose, expressed as time

Which exercise is the cheapest way to find missing steps?

Answer: A tabletop discussion

What is a stop rule?

Answer: A condition that tells the team to abort the drill and return to normal

What is the best output of a drill?

Answer: A list of surprises with owners and dates, followed by a repeat test

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. Debugging production under pressure

15 minutes. Run an incident investigation with hypotheses, one change at a time, mitigation before root cause, clear roles and notes that make the review easy.

Debugging in production differs from debugging on a laptop. Users are affected now, you cannot pause the system, and several people are looking at it at once. The goal changes: first stop the harm, then understand the cause. A calm method keeps a team from making things worse.

What you will learn

- Mitigate first, investigate second
- Working with hypotheses and one change at a time
- Roles in an incident and what to write down
- Reading recent changes as the first suspect
- Closing the loop: verification and follow-up

Mitigate, then understand

The quickest safe action usually is not the deepest fix: roll back the last deploy, turn off a feature flag, shift traffic away from a bad zone, or add capacity. Do this first if it is safe and reversible, and then investigate while users are no longer harmed.

Hypothesis loop

1. State what you see (symptom, start time, affected users).
2. List what changed near the start: deploys, flags, config, traffic, certificates, dependencies.
3. Pick the most likely and cheapest hypothesis to test.
4. Test it with a query or a read-only command.
5. Change one thing, and watch the same metric.

Write each hypothesis and the result in the incident channel. It prevents repeated work and builds the timeline for the review.

Roles

Role	Job
Incident commander	Decides, assigns work, keeps the pace; does not debug
Operations lead	Runs the commands and changes
Communications lead	Updates stakeholders and the status page on a schedule
Scribe	Keeps a timestamped log of actions and findings

On a small team one person holds two roles, but the commander should not be the one with hands on the keyboard.

Useful questions

- What changed? What is different between affected and unaffected users?
- Is it getting worse, steady or improving?
- What is the blast radius, and can we reduce it?
- What would we expect to see if this hypothesis were true? Do we see it?

```
# Compare affected and unaffected quickly
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
kubectl rollout history deploy/shop
kubectl get events -A --sort-by=.lastTimestamp | tail -20
curl -s "$STATUS_URL/api/v1/changes?since=2h" | jq '.[].summary'
```

Diagram: Incident investigation loop

Part	What it does	What goes wrong
Detect and declare	An alert or report is confirmed and an incident is declared with a commander.	Delay in declaring means uncoordinated, duplicated effort.
Mitigate	Roll back, flag off, fail over or add capacity.	Digging for root cause while users are failing prolongs harm.
Hypothesise and test	List changes, pick the likeliest, test it cheaply.	Several simultaneous changes destroy the evidence.
Verify recovery	Confirm user-facing metrics returned to normal.	Declaring victory on internal graphs while users still fail.
Blameless review	Write the timeline, causes and follow-up actions.	Skipping the review repeats the incident.

Flow: Detect and declare to Mitigate / Detect and declare to Hypothesise and test / Mitigate to Verify recovery / Hypothesise and test to Verify recovery / Verify recovery to Blameless review

Incident: three people, three changes

Incident: *Checkout latency triples at 16:20. Three engineers each make a change within five minutes.*

```
16:24 alice  restarted payments pods
16:26 bob    increased db pool 20 -> 50
16:28 chen   disabled the new recommendations flag
16:31 latency back to normal. which change fixed it?
```

Latency recovered, but nobody knew why. Bob's larger pool had also raised database load, and the problem returned the next day. In the review the team agreed on one operator making changes at a time, announced in the channel before each action, and a rule that a rollback comes before experiments.

Common mistakes

- Several people changing things at once
- No written timeline
- Skipping the check that users have recovered
- Blame, which makes people hide information

Check yourself

In an active incident with a recent deploy, what usually comes first?

Answer: A safe, reversible mitigation such as rollback

Why change one thing at a time?

Answer: So you can tell which action had which effect

What does the incident commander do?

Answer: Coordinates, decides and assigns, and does not debug

Why keep a timestamped log during an incident?

Answer: It supports handover, avoids repeated work and makes the review accurate

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 14. Senior skills: writing design docs and RFCs

12 minutes. Write short design documents that state the problem, options and trade-offs, get useful review early, and record decisions so future engineers know why.

At a senior level your output is increasingly decisions, not code. A design document is how you make a decision visible, test it against other people's knowledge and leave a record. A good one is short, specific and honest about what is uncertain.

What you will learn

- When a design doc is worth writing
- A structure that readers can scan
- How to present options and trade-offs fairly
- Running review and recording the decision

When to write one

Write a design document when a change is hard to reverse, affects several teams, has security or cost impact, or when you cannot explain the approach in a few sentences. Skip it for small, reversible changes where a pull request description is enough. The cost of a document is a day or two; the cost of a wrong design discovered after launch is months.

A structure that works

```
title: "Move session storage to a managed cache"
status: draft            # draft, in review, accepted, rejected, superseded
authors: ["A. Rao"]
reviewers: ["platform", "security", "checkout"]
sections:
  - context: "Why now. What is broken or missing. One paragraph."
  - goals: "What success looks like, with numbers where possible."
  - non_goals: "What this design deliberately does not solve."
  - proposal: "The chosen design in enough detail to build and review."
  - alternatives: "Two or three real options and why they lost."
  - risks: "What could go wrong, how we will notice, how we roll back."
  - rollout: "Steps, owners, dates, and the decision points."
  - open_questions: "What we do not know yet."
```

Start with the **context and goals**, because reviewers cannot judge a solution without the problem. State **non-goals** to stop scope creep. Put a short summary at the top that a busy executive can read in one minute. Use a diagram for flows and failure paths, and prefer concrete numbers (traffic, data size, latency, cost) over adjectives.

Options and trade-offs

The most useful section is **alternatives**. Present real options, including "do nothing" and a simpler one, with the same criteria for each: effort, risk, cost, operational burden, reversibility. Say why you prefer yours and what would change your mind. A document with one option reads as a decision already made, and reviewers disengage. Be honest about weaknesses of your own proposal; you will be trusted more.

Review and decision

Share early as a draft, ask named people for specific input ("security: does this meet the key-handling rule?"), and set a review deadline. Respond to every comment, even with "not doing this because...". Hold a short meeting only for unresolved points. When the decision is made, change the status, record the date and who decided, and link the document from the code or repository. An **architecture decision record** (a one-page summary per decision) is useful for the long term.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Senior skills: writing design docs and RFCs fits together

Part	What it does	What goes wrong
When to write one	Write a design document when a change is hard to reverse, affects several teams, has security or cost impact, or when you cannot explain the approach in a few sentences.	Writing a design after the code is done.
A structure that works	Start with the context and goals, because reviewers cannot judge a solution without the problem.	Burying the key decision in a long document.
Options and trade-offs	The most useful section is alternatives.	One option, presented as the only one.
Review and decision	Share early as a draft, ask named people for specific input ("security: does this meet the key-handling rule?"), and set a review deadline.	No named reviewers and no deadline.

Flow: When to write one to A structure that works / A structure that works to Options and trade-offs / Options and trade-offs to Review and decision

Incident: The design nobody read

Incident: *A 30-page design for a new deployment system was circulated on Friday and approved on Monday with two comments, and then a month later the on-call team refused to run it.*

```
$ wc -w deployment-system-design.md
11840 deployment-system-design.md
$ grep -c "rollback" deployment-system-design.md
2
```

The document was long, buried the key decision on page 14, had no alternatives section and no named reviewers, so the on-call team never saw it. After launch they found there was no easy rollback. The author rewrote it as a five-page document with a one-page summary, risks and rollback, asked the on-call team and security for explicit review, and the second version found three real issues before any code shipped.

Common mistakes

- Writing a design after the code is done
- One option, presented as the only one
- Burying the key decision in a long document
- No named reviewers and no deadline

Check yourself

When is a design document most valuable?

Answer: When the change is hard to reverse or affects several teams

Why list non-goals?

Answer: To prevent scope creep and set expectations

What makes the alternatives section useful?

Answer: Real options compared with the same criteria

What should happen once a decision is made?

Answer: Update the status, record who and when, and link it from the code

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 15. Senior skills: estimating and planning work

11 minutes. Give honest estimates as ranges with assumptions, break work into small slices, plan for operations and risk, and communicate changes early.

"How long will it take?" is the question every senior engineer must answer, and every estimate is partly a guess. The aim is not to be exact but to be useful: a range with stated assumptions, updated as you learn, so that others can plan around it.

What you will learn

- Why estimates fail and how to reduce the error
- Giving ranges and confidence instead of single dates
- Slicing work and finding the risky parts first
- Planning for operations, review and the unknown

Why estimates are wrong

People estimate the happy path and forget reviews, testing, deployment, waiting for others, meetings, incidents and the discovery of unknowns. They anchor on the first number they heard, and they feel pressure to give a short one. Work with new technology or unclear requirements is the most uncertain. Past data helps more than intuition: compare with similar tasks you finished and how long they really took.

Give a range and state assumptions

Say "two to four weeks, most likely three, assuming the API owner reviews within two days and we reuse the existing auth module". A range tells the listener the uncertainty. A useful technique is three points: best case, likely and worst case. Convert the range into a commitment only when risk is reduced: "I'll know more after the spike on Thursday, and then I'll give a date I'd stake my name on." Tell people when an assumption breaks, as early as you notice.

```
estimate:
  task: "Migrate billing jobs to the new scheduler"
  range_weeks: [3, 6]
  most_likely_weeks: 4
  confidence: medium
  assumptions:
    - "Scheduler team supports the retry feature by week 2"
    - "No change to the invoice format"
  risks:
    - "Hidden dependency on cron environment variables"
  first_step: "Two-day spike to list all jobs and dependencies"
  next_update: "Friday"
```

Slice and de-risk

Break the work into slices that each deliver something testable, ideally in a few days. Do the riskiest or most uncertain parts first (a spike or prototype), so bad news arrives early. Avoid "90 percent done": define done as running in production with monitoring and a documented rollback. Include time for code review, tests, documentation, migration and cleanup, and for being on call. For shared work, map dependencies on other teams and ask them for their own dates.

Plan for the unknown and say no clearly

Keep slack: a plan fully booked at 100 percent fails at the first surprise. A rough rule is to plan 60 to 70 percent of capacity for committed work, leaving the rest for incidents, support and learning. When asked to shorten a schedule, offer options instead of only refusing: reduce scope, add a phase, move the date, or add help (which has a startup cost). Make trade-offs visible: "We can ship by the 15th without the audit log, or by the 29th with it."

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Senior skills: estimating and planning work fits together

Part	What it does	What goes wrong
Why estimates are wrong	People estimate the happy path and forget reviews, testing, deployment, waiting for others, meetings, incidents and the discovery of unknowns.	Giving one number with no assumptions.
Give a range and state assumptions	Say "two to four weeks, most likely three, assuming the API owner reviews within two days and we reuse the existing auth module".	Estimating only coding time.
Slice and de-risk	Break the work into slices that each deliver something testable, ideally in a few days.	Leaving the riskiest part to last.
Plan for the unknown and say no	Keep slack: a plan fully booked at 100 percent fails at the first surprise.	Planning at 100 percent capacity.

Flow: Why estimates are wrong to Give a range and state assumptions / Give a range and state assumptions to Slice and de-risk / Slice and de-risk to Plan for the unknown and say no

Incident: The three-week migration

Incident: *An engineer estimated three weeks for moving a service to a new platform, and it took four months.*

```
$ ls jobs/ | wc -l
214
$ grep -l "legacy-env" jobs/* | wc -l
97
```

The estimate assumed 20 simple jobs, but there were 214, and nearly half read environment variables from the old platform. No one had counted them. The lead restarted with an inventory, grouped jobs by pattern, migrated the easiest group first and gave updated ranges every two weeks. The final plan was honest but late. The next project began with a two-day inventory spike, and its estimate was within 15 percent.

Common mistakes

- Giving one number with no assumptions
- Estimating only coding time
- Leaving the riskiest part to last
- Planning at 100 percent capacity

Check yourself

How should you present an estimate?

Answer: As a range with assumptions and risks

Why do the riskiest part first?

Answer: Bad news arrives early while you can still change the plan

What should be in the definition of done?

Answer: Running in production with monitoring and a rollback plan

Your manager wants the date moved earlier. What helps?

Answer: Offer scope, phase or date options and make trade-offs visible

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 16. Senior skills: running incident calls and post-mortem meetings

12 minutes. Lead a live incident call with clear roles and updates, and facilitate a blameless post-mortem meeting that produces owned, specific actions.

Technical skill finds the cause. Facilitation decides whether the group finds it quickly and learns from it afterward. Both the live incident call and the review meeting are skills you can practice.

What you will learn

- Running an incident call: roles, cadence and decisions
- Keeping the room focused and calm
- Facilitating a blameless post-mortem meeting
- Turning discussion into actions that happen

The incident call

Open with a single sentence of the problem and who is the **incident commander** (IC). The IC coordinates and decides; they do not debug. Assign a **scribe** to record a timeline, a **communications lead** to update stakeholders on a fixed cadence, and **subject experts** for each area. Use one channel for the call and one for the written record.

```
incident_call:
  opening: "SEV2: checkout errors above 5 percent since 14:05. I am IC. Priya scribes. Omar on com \
    ms."
  cadence:
    internal_update_minutes: 15
    customer_update_minutes: 30
  rules:
    - "One person speaks at a time; the IC calls on experts."
    - "Say what you are about to do before you do it."
    - "Prefer mitigation (rollback, failover, scale) over finding root cause."
    - "Hand off the IC role explicitly after 60 to 90 minutes."
  closing: "State the mitigation, the next update time and who owns follow-up."
```

Repeat key facts ("so we agree: rollback of release 482 is running, next check in five minutes"), and keep side discussions out of the main call. Do not let theories become facts: label them as hypotheses and test the cheapest first. Call a break and hand over the role when people are tired.

Calm and clear

Tone is contagious. Use plain language, avoid blame, and keep updates factual: what we know, what we do not know, what we are doing, when the next update comes. If executives join, give them a separate update channel so they do not pull the technical call off course.

The post-mortem meeting

Send the draft timeline before the meeting, so the time is spent on understanding, not on reading. The facilitator (not the person who made the change) sets ground rules: blameless, curious, focused on systems and decisions. Ask "how did this make sense at the time?" and "what would have made it easier?" instead of "who caused it". Walk through the timeline, mark moments of detection, decision and recovery, and ask where information was missing. Capture contributing factors, not a single root cause.

Actions that happen

Aim for a few, specific, owned actions, each with a date and a way to verify: "Add an alert on queue age above 5 minutes, owner Omar, by 30 April" is good; "improve monitoring" is not. Classify them as prevent, detect, mitigate or learn. Track them in the team's normal tracker and review completion at the next meeting. A post-mortem whose actions are not done is theatre.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Senior skills: running incident calls and... fits together

Part	What it does	What goes wrong
The incident call	Open with a single sentence of the problem and who is the incident commander (IC).	Treating theories as facts.
Calm and clear	Tone is contagious. Use plain language, avoid blame, and keep updates factual: what we know, what we do not know, what we are doing, when the next update comes.	No clear IC, or an IC who also debugs.
The post-mortem meeting	Send the draft timeline before the meeting, so the time is spent on understanding, not on reading.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Actions that happen	Aim for a few, specific, owned actions, each with a date and a way to verify: "Add an alert on queue age above 5 minutes, owner Omar, by 30 April" is good; "improve...	Blaming people in the review.

Flow: The incident call to Calm and clear / Calm and clear to The post-mortem meeting / The post-mortem meeting to Actions that happen

Incident: The call with seven voices

Incident: *A major outage lasted 90 minutes, and the review found the fix was available at minute 20.*

```
$ grep -c "I think" incident-call-transcript.txt
64
$ grep -n "rollback" incident-call-transcript.txt | head -3
12: maybe rollback?
41: rollback might help
96: ok doing the rollback
```

Nobody was IC, several people investigated the same hypothesis, and the rollback suggestion was lost in the chatter. The company introduced the IC role, a written scribe timeline and a rule: if a recent change might be the cause, rolling it back is tried first unless there is a reason not to. The next outage with a bad release was fixed in 12 minutes.

Common mistakes

- No clear IC, or an IC who also debugs
- Treating theories as facts
- Blaming people in the review
- Vague actions without owners or dates

Check yourself

What is the incident commander's main job?

Answer: Coordinate people and decisions, not debug

Why favor mitigation over root cause during an incident?

Answer: Restoring service comes before full understanding

Which question fits a blameless review?

Answer: "How did this decision make sense at the time?"

What makes an action item good?

Answer: Specific, with an owner, a date and a way to verify

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 17. Senior skills: mentoring and growing other engineers

11 minutes. Help engineers grow through good questions, deliberate delegation, useful feedback and code and design reviews that teach, without doing their work for them.

Your impact as a senior engineer is measured partly by how much stronger the people around you become. Mentoring is not giving answers faster; it is helping others build judgment so they need you less over time.

What you will learn

- Setting up a mentoring relationship with a clear goal
- Teaching through questions, pairing and stretch work
- Giving feedback that is specific and kind
- Reviews and delegation as growth tools

Start with goals

In the first conversation ask what the person wants in the next six to twelve months: a skill, a role, more scope, a certification. Agree on how often to meet (30 minutes every week or two is plenty), what each of you will prepare, and how you will know it works. Mentoring is a relationship with a purpose, not a social chat, though trust matters. Different people need different support: new joiners need orientation, mid-level engineers need scope and feedback, senior engineers need sponsorship and strategic perspective.

Teach with questions and pairing

When someone brings a problem, resist solving it immediately. Ask: "What have you tried? What do you expect to see? What would you check next?" The goal is to model a debugging method, not to display your knowledge. Pair on tasks and let the mentee drive, with you narrating your reasoning when you do take over. Share your mistakes and how you recovered, since it shows that errors are normal and fixable. Point to documents and tools, then ask them to improve the document afterwards.

Stretch and delegate

Growth happens just beyond comfort. Give a task that is slightly bigger than the last one, with clear outcomes, boundaries and checkpoints, and stay available. Delegation is more than handing off work: explain the why, agree on what "good" looks like, agree when to check in, and let them own decisions within the boundaries. If the result is different from what you would do but works, accept it. Take the blame for failures publicly and share the credit for successes.

Feedback that works

Make it timely, specific, about behavior and impact, and offer a path forward. Use a simple pattern: situation, behavior, impact, next step.

```
feedback:
  situation: "In yesterday's incident call"
  behavior: "You posted the rollback plan in the channel before asking the IC"
  impact: "The IC lost track of who owned the step and two people ran it"
  next_step: "Say the plan out loud to the IC first, then post it. Let's practice in the next game \
     day."
  also_good: "Your timeline notes were the clearest I have seen."
```

Give positive feedback with the same specificity, since people repeat what is named. Ask for feedback on yourself too.

MARGA by EknathaLabs marga.eknathalabs.com

Reviews that teach

In code and design reviews, separate must-fix from suggestions and learning notes, explain the reason ("this retry has no backoff, which can overload the service during an outage"), and ask questions ("what happens when the queue is full?"). Link to material. Praise good decisions. A review that only lists defects teaches fear; one that explains principles teaches judgment.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Senior skills: mentoring and growing other... fits together

Part	What it does	What goes wrong
Start with goals	In the first conversation ask what the person wants in the next six to twelve months: a skill, a role, more scope, a certification.	Vague feedback ("do better") or only negative feedback.
Teach with questions and pairing	When someone brings a problem, resist solving it immediately.	Solving everything yourself.
Stretch and delegate	Growth happens just beyond comfort. Give a task that is slightly bigger than the last one, with clear outcomes, boundaries and checkpoints, and stay available.	Delegating a task without context or boundaries.
Feedback that works	Make it timely, specific, about behavior and impact, and offer a path forward.	Reviews that only list defects.
Reviews that teach	In code and design reviews, separate must-fix from suggestions and learning notes, explain the reason ("this retry has no backoff, which can overload the service during...	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Start with goals to Teach with questions and pairing / Teach with questions and pairing to Stretch and delegate / Stretch and delegate to Feedback that works / Feedback that works to Reviews that teach

Incident: The helpful rescuer

> **Incident:** *A senior engineer fixed every problem that a new teammate brought, and after a year the teammate still could not handle an on-call shift alone.*

```
$ git shortlog -sn --since="1 year ago" -- services/billing | head -3
   412 senior.engineer
    38 new.teammate
```

The senior engineer worked quickly and meant well, but the teammate never practiced. When the senior left on holiday, the team had a billing incident and nobody dared to act. In the follow-up they set up a plan: the teammate shadowed on-call, then led with the senior as backup, then took a shift alone with a written escalation path. Within three months the teammate handled incidents calmly, and the senior had time for larger design work.

Common mistakes

- Solving everything yourself
- Vague feedback ("do better") or only negative feedback
- Delegating a task without context or boundaries
- Reviews that only list defects

Check yourself

What is a good first step in a mentoring relationship?

Answer: Agree on the mentee's goals and meeting rhythm

MARGA by EknathaLabs marga.eknathalabs.com

Why ask questions instead of giving the answer at once?

Answer: It builds the mentee's own debugging method

Which feedback is most useful?

Answer: Specific behavior, its impact and a next step

How should you delegate a stretch task?

Answer: Explain why, define good, set checkpoints and let them decide within bounds

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 18. Senior skills: influencing without authority and managing stakeholders

11 minutes. Get cross-team work done by understanding what others need, building a case with evidence, negotiating trade-offs and keeping stakeholders informed without surprises.

Most important reliability and platform work crosses team boundaries, and you rarely control the people whose cooperation you need. Influence is the ability to move work forward through trust, clarity and shared goals rather than through rank.

What you will learn

- Map stakeholders and what each one cares about
- Build a case with evidence and options
- Negotiate and say no with alternatives
- Keep people informed, and handle conflict calmly

Map the people

For any initiative, list who is affected, who decides, who can block and who can help. For each one, note what they want and what they fear: a product manager wants delivery dates, a security lead wants fewer exposed credentials, an on-call engineer wants fewer pages, finance wants predictable cost. Your proposal should answer each one's concern in their language. Talk to important stakeholders before any meeting, so nobody is surprised publicly.

```
stakeholders:
  - name: "Checkout team lead"
    role: "can block"
    wants: "No change freeze during the sale"
    worry: "Migration risk before peak season"
    plan: "Offer rollout after the sale and a canary on the staging flow"
  - name: "Security"
    role: "decides"
    wants: "Short-lived credentials"
    worry: "Exceptions that become permanent"
    plan: "Show audit logs and expiry enforced by the platform"
  - name: "Finance partner"
    role: "informed"
    wants: "Forecast within 10 percent"
    worry: "Surprises in the bill"
    plan: "Monthly cost note with unit costs"
```

Build the case

Lead with the problem and its cost to the listener: pages per week, hours lost, risk to revenue, audit findings. Use data from your own systems, one clear chart or number, not ten. Present options with trade-offs and your recommendation, and ask for a specific decision ("I need approval for two engineers for one quarter"). Start with a small pilot that proves value and lowers the risk of saying yes. Make the first step easy and reversible.

Negotiate and decline well

Understand the interest behind a demand: "we need this by Friday" may really mean "we have a customer demo". Look for options that serve the interest at a lower cost. When you must say no, say it early, explain the constraint and offer an alternative: "We cannot build the full tool this quarter. We can give you a script and a runbook now, and schedule the tool for next quarter." Be consistent: use the same criteria for everyone's requests, such as risk, user impact and effort, and show the queue so people see the trade-off.

MARGA by EknathaLabs marga.eknathalabs.com

No surprises

Share bad news early with your plan: "The migration is two weeks late because of X. Here are the options." Send short, regular updates in a stable format (status, risks, decisions needed). Write down agreements and send a summary after meetings. In conflicts, separate the people from the problem, restate the other side's view fairly before disagreeing, and escalate only after trying direct conversation, and with the other party informed.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Senior skills: influencing without authority and... fits together

Part	What it does	What goes wrong
Map the people	For any initiative, list who is affected, who decides, who can block and who can help.	Surprising stakeholders with bad news late.
Build the case	Lead with the problem and its cost to the listener: pages per week, hours lost, risk to revenue, audit findings.	Describing your needs, not theirs.
Negotiate and decline well	Understand the interest behind a demand: "we need this by Friday" may really mean "we have a customer demo".	Springing proposals on people in meetings.
No surprises	Share bad news early with your plan: "The migration is two weeks late because of X.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Map the people to Build the case / Build the case to Negotiate and decline well / Negotiate and decline well to No surprises

Incident: The mandate that was ignored

> **Incident:** *A platform team announced a mandatory new CI template by email, and six months later only three of forty teams had adopted it.*

```
$ grep -l "legacy-ci-template" teams/*/pipeline.yaml | wc -l
37
```

The announcement described what the platform team wanted, not what teams gained, and ignored the teams' real obstacles: special build steps and no time for migration. After interviews, the platform team added support for custom steps, offered migration help and a clear benefit (builds 40 percent faster in a pilot), and published a dashboard of adoption without shaming anyone. Within a quarter thirty teams had moved voluntarily.

Common mistakes

- Springing proposals on people in meetings
- Describing your needs, not theirs
- Saying no without an alternative
- Surprising stakeholders with bad news late

Check yourself

Why talk to stakeholders before a meeting?

Answer: To learn their concerns and avoid public surprises

What is a strong way to present a proposal?

Answer: Problem and cost, options, a recommendation and a clear decision request

How should you decline a request?

Answer: Early, with the reason and an alternative

MARGA by EknathaLabs marga.eknathalabs.com

A stakeholder resists change. What first?

Answer: Find out their concern and address it

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 19. Senior skills: interviewing and hiring as the interviewer

11 minutes. Run fair, structured technical interviews for platform and SRE roles with clear criteria, good questions, calibrated scoring and a respectful candidate experience.

Hiring is one of the highest-leverage things a senior engineer does. A weak process rejects good people and admits poor fits, and costs the team for years. Structure, shared criteria and respect for the candidate make interviews both fairer and more accurate.

What you will learn

- Define what the role needs before you ask a question
- Design questions that reveal how a person works
- Score against criteria and avoid common biases
- Keep the candidate experience professional

Start with the role

Write what success looks like at six and twelve months, and turn it into three to five **competencies**: for example troubleshooting, system design, automation, communication and incident judgment, with expected depth for the level. Each interviewer owns different competencies, so the panel covers the role without repeating questions. Agree on what a strong, acceptable and weak answer looks like **before** the interviews.

Questions that show how someone thinks

Prefer realistic scenarios over trivia: "A service's latency doubled at 14:00. Walk me through what you do." Listen for method (scope the problem, form hypotheses, check the cheapest signals first, communicate), not for a single correct answer. Use follow-up probes to go deeper: "What would you look at next if the CPU is fine?" For design, give a small, concrete problem and let the candidate ask questions. For hands-on skill, a short realistic exercise (read a log, fix a failing script) beats whiteboard puzzles. Ask about past incidents: what they did, what they would change, what they learned. Accept different valid approaches.

```
scorecard:
  role: "Platform engineer, L4"
  competencies:
    troubleshooting:
      strong: "Forms hypotheses, tests cheaply, narrows quickly, explains reasoning"
      acceptable: "Reaches the cause with some guidance"
      weak: "Guesses randomly or changes many things at once"
    communication:
      strong: "Clear, checks understanding, states uncertainty"
      weak: "Vague, defensive, or unable to explain choices"
  rule: "Write notes during the interview; score within 24 hours; do not discuss before submitting \
    "
```

Score fairly

Take notes of what was said and done, not impressions. Submit your score **before** talking to other interviewers, to avoid anchoring on the loudest opinion. Watch for biases: liking people who resemble you, favoring a famous employer, judging confidence over competence, and the "halo" from one great answer. Ask "what evidence do I have?" for each competency. Decide on the evidence against the criteria, and give a clear recommendation with reasons. Review outcomes later: do your scores predict performance?

MARGA by EknathaLabs
marga.eknathalabs.com

The candidate's experience

Be on time, introduce yourself and the format, explain what you will do and leave time for their questions. Give hints when someone is stuck, since you are measuring ability, not the ability to cope with stress. Do not ask illegal or irrelevant personal questions. Give feedback quickly, and be kind and honest in rejections. Candidates who feel respected speak well of the company even when they do not join, and your process is the first view they get of how the team works.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Senior skills: interviewing and hiring as the... fits together

Part	What it does	What goes wrong
Start with the role	Write what success looks like at six and twelve months, and turn it into three to five competencies: for example troubleshooting, system design, automation...	Discussing before submitting scores.
Questions that show how someone	Prefer realistic scenarios over trivia: "A service's latency doubled at 14:00.	Judging on trivia and gut feeling.
Score fairly	Take notes of what was said and done, not impressions.	Submit your score before talking to other interviewers, to avoid anchoring on the loudest opinion.
The candidate's experience	Be on time, introduce yourself and the format, explain what you will do and leave time for their questions.	A poor candidate experience.

Flow: Start with the role to Questions that show how someone / Questions that show how someone to Score fairly / Score fairly to The candidate's experience

Incident: The brilliant jerk who nearly got hired

> **Incident:** *A candidate impressed one interviewer so much with trivia that the panel overlooked repeated signs of poor collaboration in other rounds.*

```
$ grep -i "talked over\|dismissive\|would not explain" interview-notes/*.txt | wc -l
5
$ grep -c "strong hire" interview-notes/*.txt
1
```

Five separate interviewers had written concerns about collaboration, but the first interviewer's enthusiasm dominated the debrief. The hiring manager noticed the pattern in the notes and asked the panel to weigh evidence against the written criteria. The team passed on the candidate. Afterward they added collaboration as an explicit competency with its own scoring, and made the independent-scoring rule mandatory.

Common mistakes

- Asking whatever comes to mind
- Judging on trivia and gut feeling
- Discussing before submitting scores
- A poor candidate experience

Check yourself

Why define a scorecard before the interviews?

Answer: So everyone judges against the same criteria

Which question type reveals working style best?

Answer: A realistic scenario with follow-up probes

Why submit scores before the group discussion?

Answer: To avoid anchoring on the first or loudest opinion

What should you do if a candidate is stuck?

Answer: Offer a hint and see how they use it

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: SRE practices

SLO maths, error budget policy, incident roles and updates, postmortem template, toil and reliability patterns.

Definitions

Term	Meaning
SLI	A measurement: good events divided by valid events
SLO	Internal target for an SLI over a window
SLA	Contract with consequences; looser than the SLO
Error budget	1 minus the SLO

Downtime allowed per 30 days

SLO	Allowed downtime
99%	about 7 h 12 min
99.9%	about 43 min
99.95%	about 21 min
99.99%	about 4 min

A 30-day window is 43,200 minutes. Multiply by the error budget fraction.

Error budget policy

Budget state	Action
Healthy	Ship features, run experiments
Under 25% left	Stricter reviews, smaller releases
Exhausted	Pause risky releases; prioritize reliability work

Agree the policy before you need it. Never using the budget also means the target may be too strict.

Burn rate

Burn rate	Meaning	Typical action
1	Budget lasts exactly the window	Normal
6	Budget gone in about 5 days	Ticket
14	Budget gone in about 2 days	Page

Alert on a long and short window together so you are paged for sustained burn only.

Incident roles

Role	Job
Incident commander	Coordinates and decides; does not debug
Operations lead	Investigates and changes systems
Communications lead	Posts updates on a schedule
Scribe	Keeps the timeline

MARGA by EknathaLabs marga.eknathalabs.com

Severity

Level	Impact	Updates
SEV1	Major outage or data loss	Every 15 to 30 minutes
SEV2	Significant degradation	Every 30 to 60 minutes
SEV3	Minor, workaround exists	Working hours

Incident sequence

1. Detect and acknowledge
2. Triage: impact, scope, severity, open a channel
3. Mitigate first (roll back, fail over, disable a flag)
4. Diagnose with evidence; ask what changed
5. Resolve and confirm recovery holds
6. Schedule the postmortem

One change at a time, announced before it is made.

Status update template

```
HH:MM UTC - SEVn - Short title
Impact: who is affected and how
Status: what we know and what we are doing
Next update: HH:MM UTC
```

Postmortem template

Section	Content
Summary	What broke, how long, who was affected
Impact	Users, requests, budget consumed
Timeline	Timestamped events
Contributing factors	The chain of conditions
Went well / badly / lucky	Honest reflection
Actions	Specific, owned, dated: prevent, detect, mitigate

Blameless: ask how the system made the mistake easy, not who made it.

Toil

Toil is manual, repetitive, automatable, reactive work with no lasting value that grows with the service.

Reduce by	Example
Automating	Script or controller replaces the procedure
Removing the cause	Fix the bug instead of restarting
Self-service	Developers do it without a ticket
Stopping	Drop work that adds no value

Keep toil below about half of an engineer's time.

MARGA by EknathaLabs
marga.eknathalabs.com

Reliability patterns

Pattern	Rule
Timeouts	Every network call; shorter than the caller's deadline
Retries	Idempotent only, exponential backoff with jitter, two or three attempts, retry at one layer
Circuit breaker	Fail fast when a dependency is failing; closed, open, half-open
Bulkhead	Separate pools per dependency
Graceful degradation	Optional features fail without breaking the critical path
Load shedding	Reject excess requests quickly (429 or 503 with Retry-After)
Rate limits	Per-client so one caller cannot harm others

Game day checklist

- Hypothesis written down
- Blast radius limited and agreed
- Abort condition defined
- Quiet period, people watching
- Findings turned into tracked actions

SLO and error budget maths

```
sum(rate(http_requests_total{status!~"5.."}[30d])) / sum(rate(http_requests_total[30d]))
1 - (sum(rate(http_requests_total{status!~"5.."}[1h])) / sum(rate(http_requests_total[1h])))
(sum(rate(http_requests_total{status=~"5.."}[1h])) / sum(rate(http_requests_total[1h]))) / 0.001

99.9 percent over 30 days   = 43.2 minutes of error budget
99.95 percent over 30 days  = 21.6 minutes
99.99 percent over 30 days  = 4.3 minutes
burn rate 14.4 for 1 hour   = 2 percent of the monthly budget gone
burn rate 6 for 6 hours     = 5 percent of the monthly budget gone
```

Warning: *An SLO of 100 percent is not a goal: it leaves no room for change. Pick a target that users notice, and write down what happens when the budget is gone.*

Warning: *Measure the SLI where users are (load balancer, synthetic probe), not only inside the service. A healthy process behind a broken proxy still looks fine from the inside.*

Incident commands and habits

```
date -u
kubectl rollout history deploy/api -n shop
kubectl rollout undo deploy/api -n shop
kubectl get events -A --sort-by=.lastTimestamp | tail -30
curl -s -o /dev/null -w '%{http_code} %{time_total}\n' https://example.com/healthz
journalctl -u api --since "30 min ago" -p warning

1. Declare, name an incident commander, open one channel.
2. Post the first update within 10 minutes, then at a fixed interval.
3. Mitigate first (roll back, fail over, turn a flag off), find the cause later.
4. Write down every change with a UTC time.
5. Confirm recovery on the user-facing graph, then close.
6. Hold a blameless review within 5 working days. Track actions with an owner and a date.
```

Warning: *In an incident do not make several changes at once. You will not know which one helped, or which one made it worse.*

Warning: *Do not restart a failing service just to see: you can lose the evidence in memory. Capture logs,* `dmesg` *and thread or heap information first when it is safe.*

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *Never paste secrets or personal data into an incident channel or ticket. It is kept for a long time and many people can read it.*

Capacity and resilience checks

```
kubectl top nodes
kubectl get hpa -A
kubectl get pdb -A
df -h; df -i
free -m; vmstat 1 5
iostat -xz 1 5
curl -s localhost:9090/api/v1/query --data-urlencode 'query=predict_linear(node_filesystem_avail_b \
    ytes[6h], 7*86400)' | jq .
```

Warning: *Load and chaos tests belong in a test environment first. In production, start small, have a way to stop, have a named owner and tell the on-call team beforehand.*

Warning: *Do not remove an alert because it is noisy. Fix the cause or the threshold, and keep the page for something a human must do now.*

Warning: *Untested backups are not backups. Restore into a clean environment on a schedule and measure how long it takes.*

MARGA by EknathaLabs
marga.eknathalabs.com